

Term Project: Food Inventory Management Platform for SMBs

Submitted by:

Sirimintra Viboonkittanakorn 6481403

Course:

ICCS225 - Database Foundations

Trimester 1 Academic Year 2025-2026

Instructor:

Asst. Prof. Dr. Boonyanit Mathayomchan

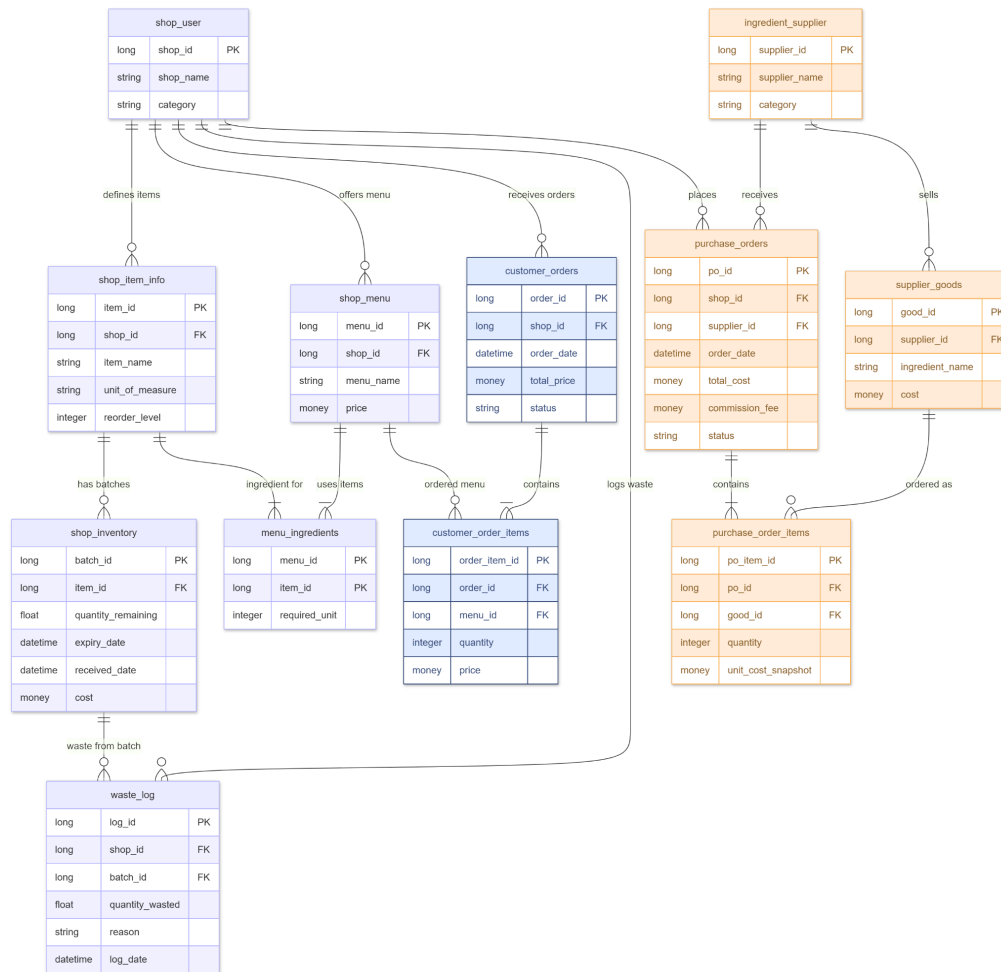
Project Scope & Pain Points

- Target: Small food & beverage businesses (Coffee Shops, Bakeries, etc.).
- Pain point:
 1. Excessive Food Waste: Many small F&B shops rely on manual tracking. As a result:
 - a. Ingredients expire unnoticed
 - b. Batches are not tracked by FIFO (first expiring = first used)
 - c. Actual consumption is unclear, causing over-purchasing
 2. Inventory Chaos:
 - a. Not knowing actual stock levels
 - b. Running out of key ingredients during peak hours
 - c. Forgetting to restock
 3. Supplier Management:
 - a. No centralized record of past orders, costs, or delivery dates
 - b. Limited supplier options
- Purpose: An inventory management platform for small F&B shops that tracks batch expiry dates (FIFO), automates restocking alerts, and facilitates supplier orders to reduce food waste, streamline inventory and supplier purchases, manage menus and sales with inventory, and find suppliers for shops.
- Pricing Model: The platform offers 2 model options
 - a. Subscription fee - 200 THB per month - For inventory management, batch tracking, expiry alerts, and menu tracking
 - b. Commission Revenue: 15% commission on each supplier purchase generated through the platform. This aligns incentives: the platform earns more when shops purchase more efficiently.

Database Design (ER Diagram)

The ER model follows five major modules:

1. Shop & inventory module: This module manages the shop's identity and its stock
 - a. Table: shop_user, shop_item_info, shop_inventory
2. Menu & recipe module: Defines what the shop sells and how it is made.
 - a. Table: shop_menu and menu_ingredients
3. Customer sales module: Records all sales transactions made by the shop to its customers
 - a. Table: customer_orders and customer_order_items
4. Supplier management module: Facilitates the B2B marketplace where shops refill stock.
 - a. Table: ingredient_supplier, supplier_goods, purchase_orders, and purchase_order_items
5. Waste log module: Tracks food waste due to expired or damaged goods
 - a. Table: waste_log



3NF achieved: No repeating groups, no partial dependencies, every non-key depends on the key

Security & Data Integrity Implementation

To ensure the platform is secure and suitable for financial transactions, several specific design patterns were implemented to prioritize data safety over raw storage efficiency.

1. Security Implementation

Password Hashing: To protect data from unauthorized access, the shop_user table stores a password_hash. This ensures that even if the database is compromised, user credentials remain secure.

	supplier_id [PK] bigint	supplier_name character varying (255)	category character varying (100)	password_hash character varying (255)
1	1	Global Matcha Co	Tea	\$2kueThisIsAFakeHashStri...
2	2	Dairy Farm	Dairy	\$2kueThisIsAFakeHashStri...

	shop_id [PK] bigint	shop_name character varying (255)	category character varying (100)	is_active boolean	password_hash character varying (255)
1	1	The Tasty Table	Restaurant	true	\$2kueThisIsAFakeHashString
2	2	Yummy Yum	Restaurant	true	\$2kueThisIsAFakeHashString

2. Data Integrity Implementation

Soft Deletes: This implementation allows for an inactive shop without losing data in case the shop user decides not to use the platform anymore

```
ALTER TABLE shop_user ADD COLUMN is_active BOOLEAN DEFAULT TRUE;
```

Time Consistency: The database utilized TIMESTAMPTZ (Timestamp with Time Zone) for the expiry_date and order_date fields. This enforces a domain of global time accuracy, ensuring that an expiry day is calculated correctly regardless of the server's time zone.

	batch_id [PK] bigint	item_id bigint	quantity_remaining numeric (10,2)	expiry_date timestamp with time zone	received_date timestamp with time zone	cost numeric (10,2)
1	1	1	1000.00	2026-06-01 12:23:53.975786+...	2025-11-28 12:23:53.975786+...	20.00
2	2	2	5000.00	2025-12-10 12:23:53.975786+...	2025-12-02 12:23:53.975786+...	0.05
3	3	3	2000.00	2025-12-13 12:23:53.975786+...	2025-12-02 12:23:53.975786+...	0.08

Entity Integrity: Every table in the database has a BIGSERIAL PRIMARY KEY, and the table for entity identifiers enforces NOT NULL to prevent incomplete records from entering the system.

```
CREATE TABLE shop_user (  
  shop_id BIGSERIAL PRIMARY KEY,  
  shop_name VARCHAR(255) NOT NULL,  
  category VARCHAR(100)  
);
```

```
1 INSERT INTO shop_item_info (shop_id, unit_of_measure)  
2 VALUES (1, 'grams');
```

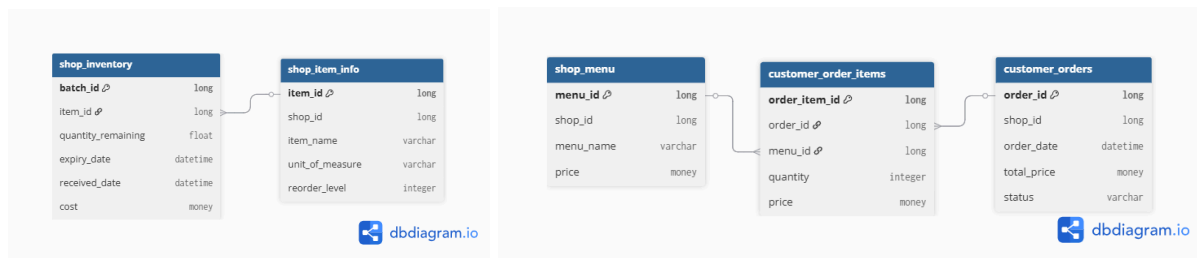
ERROR: null value in column "item_name" of relation "shop_item_info" violates not-null constraint
Failing row contains (18, 1, null, grams, 10).
SQL state: 23502
Detail: Failing row contains (18, 1, null, grams, 10).

Referential Integrity: The database uses strict FOREIGN KEY References to maintain data consistency. This ensures that every record must be linked to a valid parent entry.

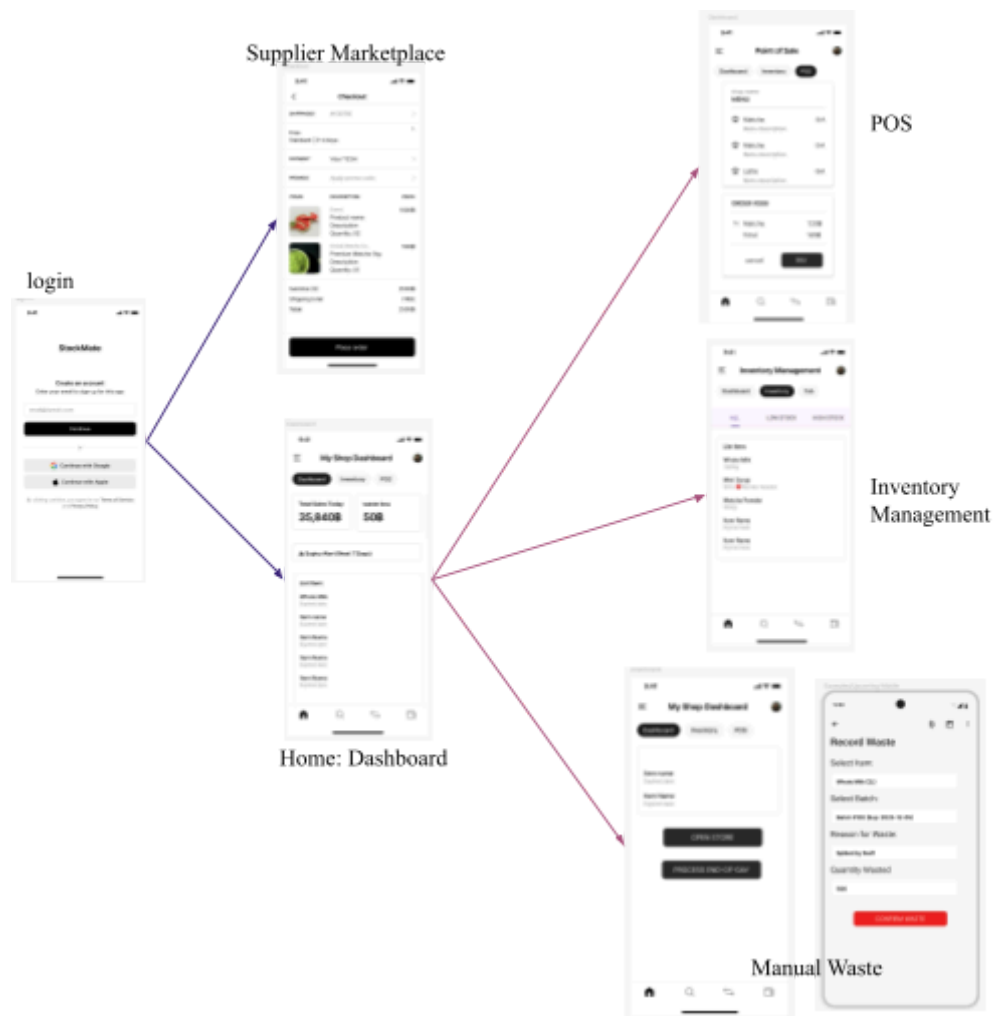
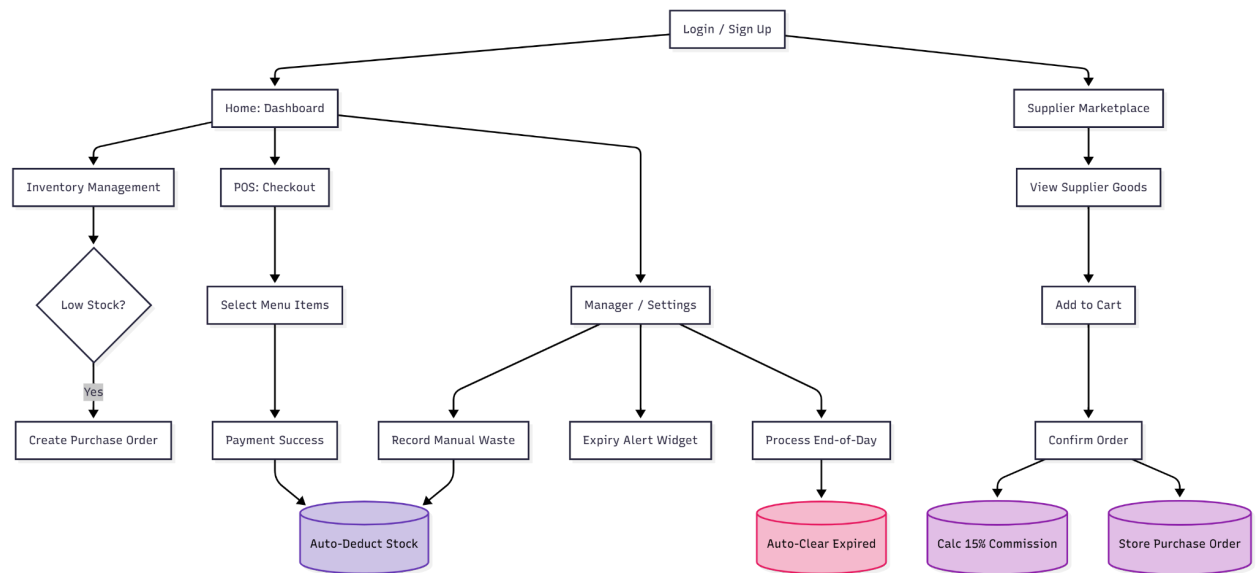
For example:

- A row in shop_inventory cannot exist unless its item_id exists in shop_item_info.
- A customer_order_item cannot be inserted unless the corresponding order_id and menu_id exist.
- A purchase_order_item must reference a valid purchase_orders entry.

The database will automatically block operations that would break the structure.

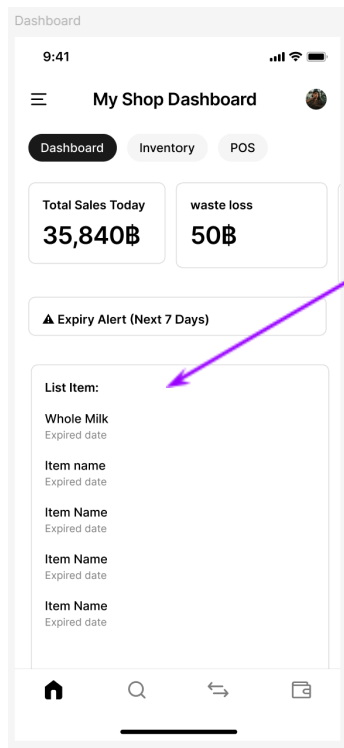


Sitemap: Platform workflow



Key Features & Function Implementation

Feature A: Waste Alert Dashboard



1

SELECT * FROM check_expiring_inventory(5, 7);

Data Output

Messages

Notifications













Showing rows: 1 to 1

Page No: 1

batch_id

item_name

expiry_date

quantity_remaining

days_until_expiry

bigint

character varying

timestamp with time zone

numeric

integer

1

2

Whole Milk

2025-12-10 12:23:53.975786+...

5000.00

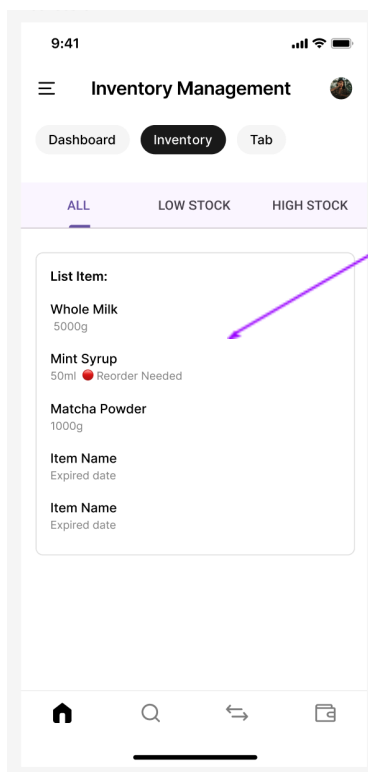
6

- Function Call:
`SELECT * FROM check_expiring_inventory(5, 7);`
- The shop owner checks the dashboard to see expiring items in 7 days to prevent food waste

```
CREATE OR REPLACE FUNCTION check_restock_level(shopID BIGINT)
RETURNS TABLE(
    item_name VARCHAR,
    total_quantity NUMERIC,
    reorder_level INTEGER,
    status VARCHAR
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        i.item_name,
        COALESCE(SUM(inv.quantity_remaining), 0) AS total_quantity,
        i.reorder_level,
        'Restock Needed'::VARCHAR AS status
    FROM shop_item_info i
    LEFT JOIN shop_inventory inv ON i.item_id = inv.item_id
    WHERE i.shop_id = shopID
    GROUP BY i.item_id, i.item_name, i.reorder_level
    HAVING COALESCE(SUM(inv.quantity_remaining), 0) <= i.reorder_level;
END;
```

```
CREATE OR REPLACE FUNCTION check_expiring_inventory(shopID BIGINT, thresholdDate INT)
RETURNS TABLE(
    batch_id BIGINT,
    item_name VARCHAR,
    expiry_date TIMESTAMPTZ,
    quantity_remaining NUMERIC,
    days_until_expiry INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT
        i.item_name,
        inv.batch_id,
        inv.expiry_date,
        inv.quantity_remaining,
        EXTRACT (DAY FROM (inv.expiry_date - NOW()))::INTEGER
    FROM shop_inventory inv
    JOIN shop_item_info i on inv.item_id = i.item_id
    WHERE inv.expiry_date BETWEEN NOW() AND (NOW() + (thresholdDate || 'days')::INTERVAL)
    AND inv.quantity_remaining > 0
    AND i.shop_id = shopID
    ORDER BY inv.expiry_date ASC;
END;
```

Feature B: Restock Alert



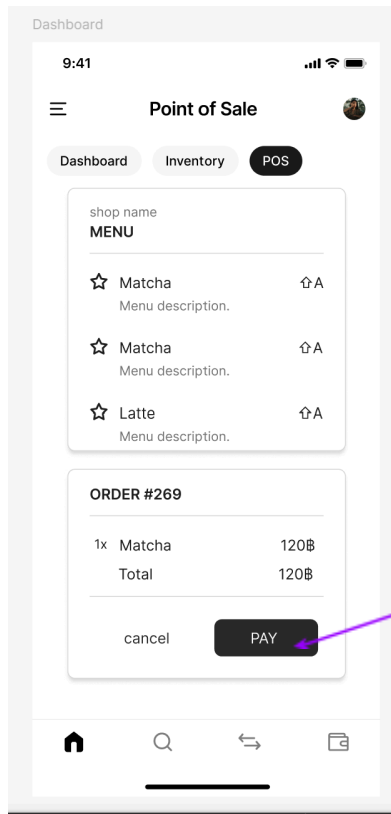
```
Query History
```

```
1 SELECT * FROM check_restock_level(5);
```

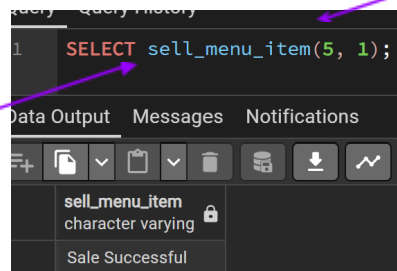
item_name	total_quantity	reorder_level	status
Mint Syrup	50.00	500	Restock Needed
Brown Sugar	0	100	Restock Needed
Simple Syrup	150.00	200	Restock Needed

- Function Call: `SELECT * FROM check_restock_level(5);`
- The shop manager checks inventory to see what needs to be reordered.

Feature C: Point of Sale



- A staff member processes a customer sale.
- Function Call: `SELECT sell_menu_item(5, 5);`
- The function looks up the recipe, then finds the oldest available batch and deducts the ingredient quantity



```
CREATE OR REPLACE FUNCTION sell_menu_item (shopID BIGINT, menuID BIGINT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    ingred RECORD;
    batch RECORD;
    required_qty NUMERIC;
    menu_price NUMERIC;
BEGIN
    SELECT price INTO menu_price FROM shop_menu WHERE menu_id = menuID;
    FOR ingred IN
        SELECT item_id, required_unit
        FROM menu_ingredients
        WHERE menu_id = menuID
    LOOP
        required_qty := ingred.required_unit;

        FOR batch IN
            SELECT batch_id, quantity_remaining
            FROM shop_inventory
            WHERE item_id = ingred.item_id
            AND quantity_remaining > 0
            ORDER BY received_date ASC -- Oldest First
        LOOP
            -- Enough Ingredient
            IF batch.quantity_remaining >= required_qty THEN
                UPDATE shop_inventory
                SET quantity_remaining = quantity_remaining - required_qty
                WHERE batch_id = batch.batch_id;
                required_qty := 0;
                EXIT;
            ELSE
                UPDATE shop_inventory
                SET quantity_remaining = 0
                WHERE batch_id = batch.batch_id;
                required_qty := required_qty - batch.quantity_remaining;
            END IF;
        END LOOP;
        IF required_qty > 0 THEN
            RAISE EXCEPTION 'Not enough stock for Ingredient ID %', ingred.item_id;
        END IF;
    END LOOP;
    INSERT INTO customer_orders (shop_id, total_price, status)
    VALUES (shopID, menu_price, 'paid');
    RETURN 'Sale Successful';
END;
$$
```

Inventory before run function

	item_name character varying	batch_id bigint	quantity_remaining numeric	expiry_date timestamp with time zone
1	Matcha Powder	1	1000.00	2026-06-01 12:23:53.975786+...
2	Whole Milk	2	5000.00	2025-12-10 12:23:53.975786+...
3	Oat Milk	3	2000.00	2025-12-13 12:23:53.975786+...
4	Cocoa Powder	4	2000.00	2026-12-03 12:23:53.975786+...
5	Mint Syrup	5	50.00	2026-03-03 12:23:53.975786+...
6	Arabica Beans	6	3000.00	2026-01-02 12:23:53.975786+...
7	Simple Syrup	7	150.00	2026-12-03 13:05:01.770595+...

Inventory after run function

	item_name character varying	batch_id bigint	quantity_remaining numeric	expiry_date timestamp with time zone
	Oat Milk	3	2000.00	2025-12-13 12:23:53.975786+...
	Cocoa Powder	4	2000.00	2026-12-03 12:23:53.975786+...
	Mint Syrup	5	50.00	2026-03-03 12:23:53.975786+...
	Arabica Beans	6	3000.00	2026-01-02 12:23:53.975786+...
	Matcha Powder	1	995.00	2026-06-01 12:23:53.975786+...
	Whole Milk	2	4850.00	2025-12-10 12:23:53.975786+...
	Simple Syrup	7	130.00	2026-12-03 13:05:01.770595+...

- The items were reduced based on the use for customer orders

Feature D: Supplier Marketplace

- Function Call: `SELECT start_purchase_order(5, 1);`

When a shop_user starts placing an order, the system creates a "Draft" order

Checkout

9:41

SHIPPINGID #158756

Free Standard | 3-4 days

PAYMENT Visa *1234

PROMOS Apply promo code

ITEMS	DESCRIPTION	PRICE
	Brand	1000B
	Global Matcha Co. Premium Matcha 1kg	500B
Subtotal (3)		2500B
Shipping total		FREE
Total		2500B

Place order

```
1 SELECT start_purchase_order(5, 1);
```

Data Output Messages Notifications

shop_id	supplier_id	total_amount	commission_fee	status
5	1	2500.00	375.00	ordered

```
CREATE OR REPLACE FUNCTION create_purchase_order(shopID BIGINT, supplierID BIGINT, totalCost NUMERIC)
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
    new_po_id BIGINT;
    commission_rate NUMERIC := 0.15; -- 15% where the app takes money
    calculated_fee NUMERIC;
BEGIN
    calculated_fee := totalCost * commission_rate;
    INSERT INTO purchase_orders (
        shop_id,
        supplier_id,
        total_amount,
        commission_fee,
        status
    )
    VALUES (
        shopID,
        supplierID,
        totalCost,
        calculated_fee,
        'ordered'
    );
    RETURNING po_id INTO new_po_id;
    RETURN new_po_id;
END;
```

Add items to the purchase order

```
Query Query History
1 SELECT add_purchase_item(12, 1, 1, 500.00);
2 SELECT add_purchase_item(12, 2, 2, 1000.00);
```

```
CREATE OR REPLACE FUNCTION finalize_purchase_order(poID BIGINT)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    real_total NUMERIC;
    comm_rate NUMERIC := 0.15;
    calc_fee NUMERIC;
BEGIN
    SELECT SUM(quantity * unit_cost_snapshot)
    INTO real_total
    FROM purchase_order_items
    WHERE po_id = poID;
    IF real_total IS NULL THEN
        real_total := 0;
    END IF;
    calc_fee := real_total * comm_rate;
    UPDATE purchase_orders
    SET total_amount = real_total,
        commission_fee = calc_fee,
        status = 'ordered'
    WHERE po_id = poID;
    RETURN 'Order Finalized. Total: ' || real_total || '. Fee: ' || calc_fee;
END;
```

```
1 SELECT finalize_purchase_order(12);
```

Query Query History

```
1 SELECT * FROM purchase_orders WHERE po_id = 12;
```

Data Output Messages Notifications

po_id [PK] bigint	shop_id bigint	supplier_id bigint	order_date timestamp with time zone	total_amount numeric (10,2)	commission_fee numeric (10,2)	status character varying (50)
12	5	1	2025-12-03 22:13:40.662108+	2500.00	375.00	ordered

- Function Call: `SELECT finalize_purchase_order(12);`
- This function sums the items, calculates the 15% Commission, and updates the status to 'ordered'.

Feature E: Manual Waste Recording

Examples/Upcoming-Mobile

9:30

Record Waste

Select Item:

Whole Milk (2L)

Select Batch:

Batch #102 (Exp: 2025-12-05)

Reason for Waste:

Spilled by Staff

Quantity Wasted

500

CONFIRM WASTE

- Adjust stock when the ingredients are immediately wasted. For example, the staff accidentally spills milk.
- The function targets that specific batch, moves the quantity to the waste_log table, and updates the inventory to the current amount.

```
SELECT record_waste(2, 10, 20, 'Spilled');
```

Query Query History

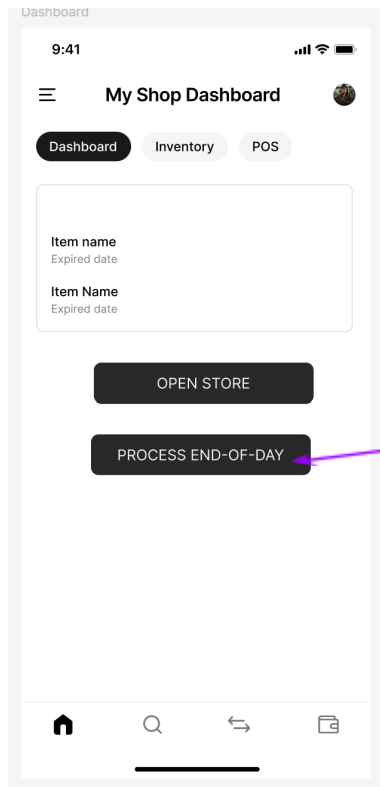
```
1 SELECT * FROM waste_log WHERE shop_id = 2;
```

Data Output Messages Notifications

log_id [PK] bigint	shop_id bigint	batch_id bigint	quantity_wasted numeric (10,2)	reason character varying (255)	log_date timestamp with time zone
1	1	2	17	Dropped	2025-12-03 21:46:23.289355+
2	2	2	16	Expired	2025-12-02 21:46:23.289355+
3	7	2	15	Sample Testing	2025-12-03 21:46:23.289355+
4	8	2	14	Expired	2025-12-01 21:46:23.289355+
5	9	2	10	Spilled	2025-12-03 22:34:31.27878400

```
CREATE OR REPLACE FUNCTION record_waste(
    shopID BIGINT,
    batchID BIGINT,
    amount NUMERIC,
    reason VARCHAR
)
RETURNS VARCHAR
LANGUAGE plpgsql
AS $$
DECLARE
    current_qty NUMERIC;
BEGIN
    SELECT quantity_remaining INTO current_qty
    FROM shop_inventory
    WHERE batch_id = batchID;
    IF current_qty IS NULL THEN
        RAISE EXCEPTION 'Batch ID % not found.', batchID;
    END IF;
    IF amount > current_qty THEN
        RAISE EXCEPTION 'Cannot waste % units. Only % units available in Batch %.', amount, current_qty, batchID;
    END IF;
    INSERT INTO waste_log(shop_id, batch_id, quantity_wasted, reason, log_date)
    VALUES (shopID, batchID, amount, reason, NOW());
    UPDATE shop_inventory
    SET quantity_remaining = quantity_remaining - amount
    WHERE batch_id = batchID;
    RETURN 'Success: Wasted ' || amount || ' units. Remaining: ' || (current_qty - amount);
END;
```


Feature F: Admin End-of-Day Process



- Function Call: `SELECT process_daily_expiry(5);`
- The shop manager closes the shop and ensures that expired items are removed from the active inventory and recorded to the waste log.
- The function loops through the entire inventory. It identifies any batch that has an expiry date that is less than today's date, logs it as waste, and automatically clears the stock.

```
SELECT process_daily_expiry(5);
```

```
CREATE OR REPLACE FUNCTION process_daily_expiry(shopID BIGINT)
RETURNS TABLE (
    result_message VARCHAR
)
LANGUAGE plpgsql
AS $$
DECLARE
    _batch RECORD;
    _count INTEGER := 0;
BEGIN
    FOR _batch IN
        SELECT shop_inventory.batch_id
        FROM shop_inventory
        JOIN shop_item_info ON shop_inventory.item_id = shop_item_info.item_id
        WHERE shop_inventory.expiry_date < NOW()
        AND shop_inventory.quantity_remaining > 0
        AND shop_item_info.shop_id = shopID
    LOOP
        PERFORM record_waste(shopID, _batch.batch_id, 'Auto-Expired');
        _count := _count + 1;
    END LOOP;

    RETURN QUERY SELECT ('Processed ' || _count || ' expired batches.'):VARCHAR;
    -- This check end-of-day expired items
END;
$$;
```

Deliverables & Access

GitHub Repository: <https://github.com/Hamtaro-chan/fnb-inventory-management-platform>

Live Database Connection (Render):

Hostname: dpg-d4jklvndiees738rohfo-a

Port: 5432

Database: fnb_platform_db

Username: root

Password: X7pbXxNl5B0UdqpSKjQKctIkubeuFEGm

External Database URL:

postgresql://root:X7pbXxNl5B0UdqpSKjQKctIkubeuFEGm@dpg-d4jklvndiees738rohfo-a.singapore-postgres.render.com/fnb_platform_db